GameDB

1.6.0

Generated by Doxygen 1.8.14

Contents

Chapter 1

Main Page

1.1 Usage Guide

1.1.1 Overview

GameDB is a Unity Plugin that provides an easy to use and powerful game and meta data editor. With this asset your scripts no longer have to use hard-coded references and instead can refer to your database for lists of values, files, prefabs, and other data types. GameDB is designed to produce JSON and associated classes to load and access this data easily within a game. It supports many data types, relationships between tables and hot-reloading to make working with game data easy and intuitive.

Some examples of usages would be for data relating to things like Loot tables, NPC balancing data, Prefab lookups for procedural assets, game configuration, and many many more.

1.1.2 Getting Started

The GameDB Editor window is opened via navigating to *Window* > *GameDB* menu and clicking on Open Editor menu item.

To get started creating and editing a gameDB click on the *Create GameDB* button. This will ask you to select a location to save your gameDB. When you click OK this will create two files a .json and .schema.json file in the directory chosen (recommended location is within a Resources folder as using Resources.Load is one of the easier ways to access the data).

The gameDB will then be loaded and ready to edit.

The first thing to do with your new gameDB is set a Scope Name for the gameDB. This name will be appended to the namespace of the exported C# classes your gameDB creates. For example a scope name of “MyData” would produce the namespace GameDBMyData. This scope name is also used as an identifier for various other things explained in more detail later.

Next you can create tables by entering a *Table Name*, select a key type and clicking on *Create Table*. This will create a table below that is collapsed by default. Click on the table name and it will expand to show your table. By default this will be a table with no keys or fields.

To create a field click on the *Edit Table* button, then enter a *Field Name* under the *Modify Schema* section. Select a data Type, check the tick-box for whether you would like it to be an array of data and then click *Create Field*.

This will then add a field to your table that can be removed with the [x] button (only visible after clicking *Edit Table*) if required.

To add some data enter a Key in the field at the top of the table then click *Create Key*. This will create a entry in the table below where you can now enter data for each of your fields.

Once you are happy with the data you have entered you can save the gameDB by clicking on *Save GameDB*, this will serialise out the data and the schema to the location selected earlier.

Next to use the data in your game you need to export the C# classes that will allow you to access the data. To do this click *Generate Classes* and select where to export the classes. This will generate C# classes that will provide easy and strongly typed access to your data.

1.1.3 Usage In Game

You are free to use the json data however you like but the GameDB plugin generates code based on your data schema that provides an easy way to integrate the data with your game.

When you click *Generate Classes* this will generate code in the selected location that provide access to the data. Each of the classes will be namespaced with a name like GameDB{SCOPE_NAME_HERE} for example: GameDBMain. These are auto-generated classes and hand editing of the files is generally not recommended.

The main class that represents your game data is the GameDB class. This class will load the json, deserialize it and provide access to each of the loaded tables. The GameDB class also has a OnDBLoaded event that is triggered when the data is loaded.

An example of loading the game data:

```
using GameDBMain;
using UnityEngine;

public class Test : MonoBehaviour
{
    void Start ()
    {
        //A name is given when instantiating to identify the gameDB when editing during play
        GameDB gameDb = new GameDB("MyGameDB");
        gameDb.OnDBLoaded = delegate() {
            //Access game data here once the DB is loaded
        };

        //Load the json from the resources folder
        gameDb.Load("GameDBs/gameDB");
    }
}
```

A few classes are then created for each of the tables defined in your game data. There is a Table, Schema and Row class. Normally prefixed with the table name for example MyDataTable.cs, MyDataSchema.cs and MyData.cs. The Table class provides accessors for getting a particular row by Key or getting all the rows as a dictionary. The Schema class provides static accessors for the table name, field names and keys in the table. Lastly the Row class represents a particular row in your table and provides accessors for each field in the data.

An example of accessing data:

```
gameDb.OnDBLoaded = delegate() {
    //Access game data here once the DB is loaded
    var shipPart = gameDb.ShipPartsTable.GetByKey(ShipPartsSchema.KeyWing);

    Debug.Log(shipPart.CostVal);

    foreach (var loot in gameDb.LootTable.GetRows())
    {
        Debug.Log(loot.Value.NameVal);
    }
};
```

1.1.4 Editing in-game

The gameDB can be edited while in game and hot-reloaded so you can balance while playing the game. If you have the GameDB Editor window opened when you are playing the game you can load one of the gameDBs you have loaded in the game by selecting the gameDB by its name in the *Runtime GameDB* dropdown, then selecting the base gameDB it was loaded from in the *Base GameDB* dropdown then clicking *Load GameDB*.

This will load the gameDB that is in memory in the game and allow you to edit it and then cause the data to reload in-game by clicking the *Reload In-Game* button. This will cause the OnDBLoaded event to be triggered on the gameDB in the game and when this happens your game code can react to the change in anyway it likes (see advanced topics for working with hot reloading).

The game data you edit while playing the game won't be saved back to the json until you click the *Save GameDB* button, meaning you can experiment with changes while the game is running then if you stop playing the game without saving the gameDB those changes will be discarded.

1.1.5 Data Types

Currently the GameDB plugin supports the following data types:

- String
- Int32
- Float
- Bool
- UnityEngine.Color
- UnityEngine.Vector2
- UnityEngine.Vector3
- UnityEngine.Vector4
- Prefabs (Version 1.2.0 and lower)
  - Prefabs can be imported from the project and when settings data a drop down of imported prefabs will be available.
  - Prefabs must be imported from a resources folder as they are loaded via Resource.Load when accessed in game.
  - There are two accessors added into the generated Row class for each prefab field; one returning the path of the prefab and another returning the UnityEngine.Object representing the prefab.
- UnityEngine.Object (Version 1.3.0 and higher (Pro only currently))
  - Supports the loading of any unity asset from the Resources folder ie Textures, Audio, Prefabs etc
  - UnityEngine.Objects must be imported from a resources folder as they are loaded via Resource.Load when accessed in game.
  - There are two accessors added into the generated Row class for each unityObject field; one returning the path of the UnityEngine.Object and another returning the UnityEngine.Object representing the unityObject.
- Enums (Pro version only)
  - Enums are imported from game code and can be selected as a type. Then when setting data for the field a drop down is given of the enum members that can be selected.

- Table References (Pro version only)
  - Fields can be a reference to another table within the gameDB. Then when setting data for the field a drop down is given with the keys of the referenced table
  - There are two accessors added into the generated Row class for each Table Reference field; one returns the string key of the entry and the other an instance of the row referenced.

Arrays

All the above data types can be stored as arrays. To modify the values of an array click on the [E] button next to the field and a popup will show allowing you to modify the size of the array and enter values for each index.

1.1.6 Advanced Usage

1.1.6.1 Add an existing gameDB

If you have a gameDB from another project or being migrated from somewhere else (or the settings have been reset) you can load this by clicking on the *Add Existing GameDB* button. Make sure that both the .json and .schema.json files are in the same location together.

1.1.6.2 Importing Enums

To import enums you will need to click on the *Configuration* Tab at the top of the editor window and expand the *Imported Enums* section. From here you can import and remove any enum you want.

1.1.6.3 Hot Reloading

To support editing the data while playing or support updates of the game data from a server while the client is running. The GameDB has a OnDBLoaded event that is triggered when data has be Imported or Reimported.

Generally data shouldn't be directly referenced from the gameDB due to overhead with runtime conversions of types (and loading prefabs for the prefab type). Best practice is to cache the values from the gameDB into your game classes for further access.

This means that when the OnDBLoaded event is triggered again you will need to likely re-cache any data you accessed as well as reloading any systems that rely on this data.

This is quite different to how serialised fields work but it allows consistent and deliberate work flows when dealing with data edited at run-time or live updating from a server for example.

1.1.6.4 Settings

The GameDB editor creates a GameDBEditor.settings file in the same location as the plugin under the Editor folder. This file is a JSON file representing the settings that represent your workspace. This can be edited by hand in the event of settings causing any issue. This file contains the paths of the loaded gameDBs, the export path for C# classes and paths to any imported items, plus more.

Chapter 2

Google Sheets Import/Export Guide

(Pro version only)

2.1 Overview

The GameDB (Pro) plugin supports importing and exporting from/to google sheets via a self-hosted web app.

This functionality allows mainly for designers who may be used to this workflow to migrate to using the tool or allow analysis of the data after it has been exported, or can allow for migrating data that may already be stored in a spreadsheet into Unity and the plugin,

This is provided as a value added component and is not a recommended way of working with the plugin long term as certain future features may not be supported via Google Sheets Export/Import.

2.2 Setup

Found in *Assets/Plugins/GameDBLibrary/GoogleSheets* is a Google App Script file called *GoogleSheetWebApp.gs*

This file will be imported into a Google App Script project that can be created by visiting `https://www.google.com/script/start/` and clicking on the *Start Scripting* button. Copy the contents of the *GoogleSheetWebApp.gs* file into the main window (see screenshot below). Then give the project a name and save it from the file menu.

Before the script is usable it needs to be published as a wep app this can be done by select the *Publish* > *Deploy as a web app...* option from the menu. You will be asked to select who the web app operates under and who has access. You have to have it execute as a particular user as the plugin can't authenticate otherwise and you have to allow access to *Everyone, even anonymous* so be careful not to share your web app URL which you will get in the next dialog after clicking deploy.

You can then use the URL you obtained to enter into the plugin in the *Web App Url:* field when you click on the *Google Sheets* button.

2.3 Usage

With the web app setup you can now export data from Unity into a spreadsheet or import it from one.

To specify the spreadsheet to export to/import from, create a spreadsheet in Google Drive or Google Sheets directly then take note of the spreadsheet id found by getting the long alpha-numeric string similar to the one hightlighted in this example: https:/docs.google.com/spreadsheets/d/**e3jmJoPNlsumfsYjW7sGOv81taMDe3jmJoPNlsumfsYjW7sGOv81taMD**/edit

This is then entered in the *Sheet ID:* field of the dialog that is shown after clicking on the *Google Sheets* button.

2.4 Notes

- To get the best results with importing make sure to export your table first, this will create the required sheet in your document and give you an idea of the format required for import
- A Sheet will be created for each table in your gameDB
- It is recommended to use a new sheet id for each gameDB you have

Chapter 3

WIP: Server and over the air updates

(Pro version only)

3.1 Overview

The GameDB (Pro) plugin provides functionality for uploading and downloading a gameDB to/from a server. Effectively allowing over the air updates of your data without the need to republish or resubmit your app before getting an update out to users.

Included in the plugin is a server written in Golang that interfaces with AWS and S3/DynamoDB to manage the updates and the uploaded files. This server can be used as is or is provided as an example for writing your own or integrating it with other infrastructure.

The plugin also contains high level helper functions to retrieve the data but also provides APIs for doing most of the separate tasks yourself if custom integration with your app is required.

The data uploaded and downloaded from the server is both compressed and encrypted (more info HERE) and is only a diff between a base version of the data and the current revision meaning it is as small and efficient as possible. The diff format is that defined by the JSON Patch format http://jsonpatch.com/.

3.2 Setup

3.2.1 Server

The provided server is easily built and deployed as it comes with build scripts and a docker file. Extra work will be required to integrate it with pre-existing infrastructure but deployment won't be covered here, though I can be contacted to discuss options of providing support to help you deploy the server (This will be charged extra to the price of the plugin)

The server was built using golang version go1.8.3. While earlier or later versions are likely to work just fine, I suggest you use the recommended version to avoid any issues.

Some prerequisite software will need to be installed:

- golang 1.8.3 https://golang.org/dl/

- glide https://github.com/Masterminds/glide
- docker https://www.docker.com/

Within the *Assets/Plugin/GameDBLibary* folder is a *server.zip* that you should extract to your go workspace https://golang.org/doc/install.

You will then need to in the root of the server directory run the command `glide install` this will download and install all the required dependencies for the server.

After this has been completed you can proceed to build the server by running `./build.sh`. This will compile the executable and build a docker image ready to deploy or test.

Next step is to setup the AWS services you will be interfacing with. This guide won't go into detail on how to set these up but will outline the requirements (If you need extra support feel free to contact me).

S3 You will need to either create or use an existing S3 bucket. Files will be uploaded here that will have ACLs set to public-read (If you would rather these are access controlled you will need to modify the server and client to support this).

DynamoDB You will need a DynamoDB table set up that has a *Partition key* called *tag* as a string type and a *Sort key* called *scope* as a string type. Records regarding the uploaded files will be stored here.

Before testing the server a configuration file needs to be defined see configuration section below

Once you have a *conf.yaml* that suits your needs then you can start the docker container for testing by running `docker run --v ./conf.yaml:/app/conf.yaml ***TODO GET ACTUAL COMMAND***`

3.2.2 GameDB Plugin

To connect the Unity plugin to the server for uploading and managing deployed gameDBs you will need to configure the URLs for your server and for the S3 bucket you previously setup.

To do this open the GameDB Editor by going to ***Window*** > ***GameDB*** > ***Open Editor*** then in the editor window navigate to the *Deployment* tab.

There should then be a *Deploy to Server* section you can expand where you will see two entry fields for *GameDB Server Host* and *Download Server Host*. Assuming you have a server deployed or running locally you can set these then you should be able to start deploying gameDBs.

Example values for the host fields are: *GameDB Server Host*: `http://localhost:8000` (if running it locally for testing) *Download Server Host*: `https://s3-eu-west-1.amazonaws.com/mybucket`

3.2.3 Server Configuration

The server can be configured in a few different ways. These include via a conf.yaml file, or a combination of the conf.yaml and environment variables.

Below is an example conf.yaml file and then an explanation of what each value configures:

```
server:
  port: 8000               //The port the server listens on. Recommended to leave as default as the docker con
  writeTimeout: 30         //Timeout in seconds for writing a http response
  readTimeout: 30          //Timeout in seconds for reading a http request
  formBufferSize: 10485760 //Max size in bytes for uploaded POST data and parameters

aws:                       //The AWS config can also be set via environment variables listed here: http://docs
  id: ""                   //AWS Credentials ID
  secret: ""               //AWS Credentials Secret
  region: "eu-west-1"      //AWS Region for S3 bucket and DynamoDB table

gamedb:
  dynamodb:
    tableName: "gamedb"    //Name of previously created DynamoDB table in configured region. Can also be set b
  s3:
    bucket: "mybucket"     //Name of previously created S3 bucket in configured region. Can also be set by env
    storageRoot: "gamedb"  //Top level directory name to store files in, in S3 bucket. Can also be set by envi
  encryption:
    key: "somekey"         //A secure key to use for encrypting uploaded gameDB data to be downloaded by clien
    salt: "somesalt"       //A secure salt to use for encrypting uploaded gameDB data to be downloaded by clie

logging:
  level: "info"            //Log level to use can be set based on available log levels here: https://github.co
```

3.3 Usage

Deploying a gameDB {$deployingagamedb}

The GameDB Editor allows you to upload and publish the currently loaded gameDB. This makes it easy to make a change to your data and get it onto a server quickly and efficiently.

Terminology Deployment: A deployment is a set of revision for a particular gameDB scoped by the Scope Name of the uploaded gameDB and by a Tag. A deployment contains information about the current deployed version plus all past/future revisions.

Tag: A tag is an arbitrary label applied to a particular deployment of a gameDB. Generally this will be something like the client version the gameDB is intended for (ie. 1.1.3) or could be an identifier (ie. open beta). This allows different versions of the same gameDB to be deployed for different needs. If the schema for a gameDB changes so should the tag it is deployed against.

Revision: A revision is a particular incremental change of data for a particular gameDB and tag. Meaning if you make a change to the data without changing schema and you intend to update it for a already deployed gameDB it will be treated as a revision of the server. Which causes it to detect only the differences between the original uploaded gameDB for a particular gameDB/tag and store that only saving bandwidth when the client downloads the new data.

Retrieving Deployments On the *Deployment* tab of the GameDB Editor, if you expand the *Deploy to Server* section, you can click the *Retrieve Deployments* button to query the server for the current list of deployments for the current gameDB. If you have gameDBs already deployed this will update the Tag and Revision drop-downs below.

Uploading a new revision

Chapter 4

Namespace Index

4.1 Packages

Here are the packages with brief descriptions (if available):

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 7

Namespace Documentation

7.1 GameDBCodegenExample Namespace Reference

The GameDB plugin provides generated code for each GameDB and its associated tables to allow easy typed access to all your saved GameDB data.

Classes

- class ExampleTable

  Each table in the GameDB will have an associated class generated for it, that provides accessors for all the Rows or specifc rows. Table classes are generated with names that match the table name For example: {TableName}Table ie. MyDataTable where table name is "MyData"
- class GameDB

  Each GameDB will output a generated GameDB class that will allow access to the Tables within the GameDB. Because this is scoped to a custom namespace there are no conflicts between multiple GameDBs in the same project.

7.1.1 Detailed Description

The GameDB plugin provides generated code for each GameDB and its associated tables to allow easy typed access to all your saved GameDB data.

The generated classes will be contained within a namespace based on the Scope Name entered for the relevant GameDB. For example: GameDB{ScopeName} ie. GameDBMyTestScope if the Scope Name was "MyTestScope".

7.2 GameDBEditorLibrary Namespace Reference

Namespaces

Classes

- class GameDBEditor

  GameDBEditor provides static methods for working with the GameDB Editor within the Unity Editor. Allowing things such as subscribing to events (load/save/promote) and programatically working with the editor.

7.3 GameDBEditorLibrary.Properties Namespace Reference

7.4 GameDBLibrary Namespace Reference

Namespaces

Classes

- class BinaryGameDB

  BinaryGameDB provides utility methods for Serializing and Deserializing GameDBs to and from binary representations that are encrypted and compressed.
- class GameDBBase

  The base class all GameDBs inherit from. The GameDB provides methods for importing data.
- class JsonPatch

  Implements the json patch spec at `http://jsonpatch.com/` *and passes all tests* `https://github.com/json-patch/json-patch-tests/blob/master/spec_tests.json` *Allows applying a json patch to a json string.*
- class Logger

  The logger class allows internal logging in the GameDB classes to be redirected to any destination required ie. Unity console, file on disk or terminal output.
- class Remote

  The Remote class provides methods to communicate with GameDB servers.
- class RequestUpdater

  RequestUpdater is a class that allows an update method to be hooked into an update loop, to allow constant updating of a requests asynchronous handling mechanisms
- class RowBase

  This is the base class for all Rows in a GameDB's tables. It provides accessors to the values store within.
- class ServerResponse

  ServerResponse handles parsing basic responses from the GameDB server
- class Singleton

  A helper base class for creating singletons. Used internally but provided for utility reasons.

7.5 GameDBLibrary.Properties Namespace Reference

Chapter 8

Class Documentation

8.1 GameDBLibrary.BinaryGameDB Class Reference

BinaryGameDB provides utility methods for Serializing and Deserializing GameDBs to and from binary representations that are encrypted and compressed.

Static Public Member Functions

- static byte [ ] Serialize (string json, string encryptionKey, string encryptionSalt)

  Serializes the specified json into a binary blob.
- static string Deserialize (byte[ ] data, string encryptionKey, string encryptionSalt)

  Deserializes the specified data in a JSON string.

8.1.1 Detailed Description

BinaryGameDB provides utility methods for Serializing and Deserializing GameDBs to and from binary representations that are encrypted and compressed.

8.1.2 Member Function Documentation

8.1.2.1 Deserialize()

```
static string GameDBLibrary.BinaryGameDB.Deserialize (
            byte [] data,
            string encryptionKey,
            string encryptionSalt )  [static]
```

Deserializes the specified data in a JSON string.

Parameters

data	The data.
encryptionKey	The encryption key.
encryptionSalt	The encryption salt.

Returns

A JSON string decompressed and decrypted from the data passed in.

Exceptions

System.Exception	Throws an exception if it fails to decrypt or decompress the json

8.1.2.2 Serialize()

```
static byte [] GameDBLibrary.BinaryGameDB.Serialize (
            string json,
            string encryptionKey,
            string encryptionSalt )  [static]
```

Serializes the specified json into a binary blob.

Parameters

json	The json.
encryptionKey	The encryption key.
encryptionSalt	The encryption salt.

Returns

A binary blob encrypted and compressed of the passed in JSON

Exceptions

System.Exception	Throws an exception if it fails to encrypt or compress the json

8.2 GameDBCodegenExample.ExampleTable Class Reference

Each table in the GameDB will have an associated class generated for it, that provides accessors for all the Rows or specifc rows. Table classes are generated with names that match the table name For example: {TableName}Table ie. MyDataTable where table name is "MyData"

Inherits GameDBLibrary.TableBase.

Public Member Functions

- Example GetByKey (string key)

  Will get a particular row in the table by Key.
- bool TryGetByKey (string key, out Example row)

  Tries to get a particular row in the table by Key.
- Dictionary< string, Example > GetRows ()

  Returns a Dictionary keyed by table key representing all the rows in the table.

8.2.1 Detailed Description

Each table in the GameDB will have an associated class generated for it, that provides accessors for all the Rows or specifc rows. Table classes are generated with names that match the table name For example: {TableName}Table ie. MyDataTable where table name is "MyData"

See also

GameDBLibrary.TableBase

8.2.2 Member Function Documentation

8.2.2.1 GetByKey()

```
Example GameDBCodegenExample.ExampleTable.GetByKey (
            string key )
```

Will get a particular row in the table by Key.

Parameters

key	The key to the required row.

Returns

An instance of the required row

Exceptions

KeyNotFoundException	Throws an exception if the key is not found in the table.

8.2.2.2 GetRows()

```
Dictionary<string, Example> GameDBCodegenExample.ExampleTable.GetRows ( )
```

Returns a Dictionary keyed by table key representing all the rows in the table.

Returns

The rows in the table.

8.2.2.3 TryGetByKey()

```
bool GameDBCodegenExample.ExampleTable.TryGetByKey (
            string key,
            out Example row )
```

Tries to get a particular row in the table by Key.

Parameters

key	The key to the required row.
row	The returned row if successful.

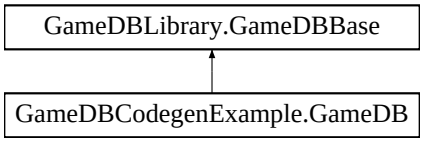
Returns

`true`/`false` indicating if the row was found.

8.3 GameDBCodegenExample.GameDB Class Reference

Each GameDB will output a generated GameDB class that will allow access to the Tables within the GameDB. Because this is scoped to a custom namespace there are no conflicts between multiple GameDBs in the same project.

Inheritance diagram for GameDBCodegenExample.GameDB:



Classes

- class UnityLogger

  (Unity only)
  By default logging by the GameDB is logged via a GameDBLibrary.Logger. This allows the used logger to be replace if logging needs to be redirected. This class redirects logging to the Unity console via Debug.Log etc To replace the Logger set GameDBBase.Logger

Public Member Functions

- GameDB (string name)

  Initializes a new instance of the GameDB class. Multiple instances of the GameDB can be loaded at run-time. The GameDB can then be identified in the GameDB Editor window by a unique name associated when the instance is created. This allows run-time balancing/modification of the DB to be scoped to a particular instance of the GameDB if for example, you don't want one characters data to change when balancing another.

- Exception Load (string path, bool notify=true)

  (Free Version only) (Unity only)
  This method will load a GameDB from the specified path. This should only be used on GameDBs stored in Unity Resources folders as it uses UnityEngine.Resources.Load If loading GameDBs from non Resources folders GameDBBase.Import(string,bool) (or other overloads) should be used instead.

- Exception Load (string path, bool notify=true, bool binary=false, string key=null, string salt=null)

  (Pro Version only) (Unity only)
  This method will load a GameDB from the specified path. The GameDB supports being compressed (via LZF) and encrypted (via AES) using the GameDBLibrary.BinaryGameDB utilities. This method will load both a binary or text based GameDB. This should only be used on GameDBs stored in Unity Resources folders as it uses UnityEngine.Resources.Load If loading GameDBs from non Resources folders GameDBBase.Import(string,bool) (or other overloads) should be used instead.

- Exception Load (string path, string language, bool notify=true, bool binary=false, string key=null, string salt=null)

  (Pro Version only) (Unity only) (Localization DB only)
  This method will load a GameDB from the specified path. This method is provided when a GameDB is marked as a LocalizationDB. It allows only a certain field (representing a language) across all tables to be loaded, saving on memory and simplyfying access to the required language. The GameDB supports being compressed (via LZF) and encrypted (via AES) using the GameDBLibrary.BinaryGameDB utilities. This method will load both a binary or text based GameDB. This should only be used on GameDBs stored in Unity Resources folders as it uses UnityEngine.Resources.Load If loading GameDBs from non Resources folders GameDBBase.Import(string,bool) (or other overloads) should be used instead.

- Exception Import (string json, string language, bool notify=true)

  (Pro Version only) (Unity only) (Localization DB only)
  Imports JSON representing the GameDB. This method is provided when a GameDB is marked as a LocalizationDB. It allows only a certain field (representing a language) across all tables to be imported, saving on memory and simplyfying access to the required language.

Properties

- ExampleTable ExampleTable `[get]`

  A getter will be generated for each table within the GameDB. This will give access to the loaded instance of a table, allowing each row and its fields to be easily access. The getter will be generated with names matching the table name. For Example: {TableName}Table ie. MyTestTable for a table name "MyTest"

Additional Inherited Members

8.3.1 Detailed Description

Each GameDB will output a generated GameDB class that will allow access to the Tables within the GameDB. Because this is scoped to a custom namespace there are no conflicts between multiple GameDBs in the same project.

See also

GameDBLibrary.GameDBBase

8.3.2 Constructor & Destructor Documentation

8.3.2.1 GameDB()

```
GameDBCodegenExample.GameDB.GameDB (
            string name )
```

Initializes a new instance of the GameDB class. Multiple instances of the GameDB can be loaded at run-time. The GameDB can then be identified in the GameDB Editor window by a unique name associated when the instance is created. This allows run-time balancing/modification of the DB to be scoped to a particular instance of the GameDB if for example, you don't want one characters data to change when balancing another.

Parameters

name	The run-time name of the GameDB shown in the GameDB editor.

8.3.3 Member Function Documentation

8.3.3.1 Import()

```
Exception GameDBCodegenExample.GameDB.Import (
            string json,
            string language,
            bool notify = true )
```

(Pro Version only) (Unity only) (Localization DB only)
Imports JSON representing the GameDB. This method is provided when a GameDB is marked as a Localization↩ DB. It allows only a certain field (representing a language) across all tables to be imported, saving on memory and simplyfying access to the required language.

Parameters

json	A string representing the JSON format of the GameDB to import
language	The name of the language field to load.
notify	if set to `true` the GameDBBase.OnDBLoaded callback will be triggered for the GameDB (defaults to `true`).

Returns

An exception is returned if the GameDB fails to imprt

8.3.3.2 Load() [1/3]

```
Exception GameDBCodegenExample.GameDB.Load (
            string path,
            bool notify = true )
```

(Free Version only) (Unity only)
This method will load a GameDB from the specified path. This should only be used on GameDBs stored in Unity Resources folders as it uses UnityEngine.Resources.Load If loading GameDBs from non Resources folders GameDBBase.Import(string,bool) (or other overloads) should be used instead.

Parameters

path	The path is relative to Application.dataPath (ie. Assets/) and does not include a file extension. For example to load Assets/Resources/GameDBs/gameDB.json use the path "GameDBs/gameDB"
notify	if set to `true` the GameDBBase.OnDBLoaded callback will be triggered for the GameDB (defaults to `true`).

Returns

8.3.3.3 Load() [2/3]

```
Exception GameDBCodegenExample.GameDB.Load (
            string path,
            bool notify = true,
            bool binary = false,
            string key = null,
            string salt = null )
```

(Pro Version only) (Unity only)
This method will load a GameDB from the specified path. The GameDB supports being compressed (via LZF) and encrypted (via AES) using the GameDBLibrary.BinaryGameDB utilities. This method will load both a binary or text based GameDB. This should only be used on GameDBs stored in Unity Resources folders as it uses UnityEngine.Resources.Load If loading GameDBs from non Resources folders GameDBBase.Import(string,bool) (or other overloads) should be used instead.

Parameters

path	The path is relative to Application.dataPath (ie. Assets/) and does not include a file extension. For example to load Assets/Resources/GameDBs/gameDB.json use the path "GameDBs/gameDB"
notify	if set to `true` the GameDBBase.OnDBLoaded callback will be triggered for the GameDB (defaults to `true`).
binary	if set to `true` the GameDB will be loaded as a binary source (defaults to `false`) (`key` and `salt` required if loading binary)
key	the key used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`
salt	the salt used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`

Returns

An exception is returned if the GameDB fails to load

8.3.3.4 Load() [3/3]

```
Exception GameDBCodegenExample.GameDB.Load (
            string path,
            string language,
            bool notify = true,
            bool binary = false,
            string key = null,
            string salt = null )
```

(Pro Version only) (Unity only) (Localization DB only)
This method will load a GameDB from the specified path. This method is provided when a GameDB is marked as a LocalizationDB. It allows only a certain field (representing a language) across all tables to be loaded, saving on memory and simplyfying access to the required language. The GameDB supports being compressed (via LZF) and encrypted (via AES) using the GameDBLibrary.BinaryGameDB utilities. This method will load both a binary or text based GameDB. This should only be used on GameDBs stored in Unity Resources folders as it uses UnityEngine.Resources.Load If loading GameDBs from non Resources folders GameDBBase.Import(string,bool) (or other overloads) should be used instead.

Parameters

path	The path is relative to Application.dataPath (ie. Assets/) and does not include a file extension. For example to load Assets/Resources/GameDBs/gameDB.json use the path "GameDBs/gameDB"
language	The name of the language field to load.
notify	if set to `true` the GameDBBase.OnDBLoaded callback will be triggered for the GameDB (defaults to `true`).
binary	if set to `true` the GameDB will be loaded as a binary source (defaults to `false`) (`key` and `salt` required if loading binary)
key	the key used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`
salt	the salt used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`

Returns

An exception is returned if the GameDB fails to load

8.3.4 Property Documentation

8.3.4.1 ExampleTable

```
ExampleTable GameDBCodegenExample.GameDB.ExampleTable  [get]
```

A getter will be generated for each table within the GameDB. This will give access to the loaded instance of a table, allowing each row and its fields to be easily access. The getter will be generated with names matching the table name. For Example: {TableName}Table ie. MyTestTable for a table name "MyTest"

The loaded instance of the associated table.

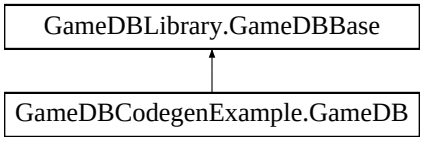
See also

GameDBCodegenExample.ExampleTable

8.4 GameDBLibrary.GameDBBase Class Reference

The base class all GameDBs inherit from. The GameDB provides methods for importing data.

Inheritance diagram for GameDBLibrary.GameDBBase:



Public Member Functions

- Exception Import (string jsonData, bool notify=true)

  Imports JSON representing the GameDB.
- Exception Import (string jsonData, string[ ] columImportList, bool notify=true)

  Imports JSON representing the GameDB. Allows specifying only certain fields to be imported. Useful for limiting the memory used of the loaded GameDB.
- RequestUpdater ImportFromServer (string serverHost, string downloadHost, string userID, string tag, string originalJSON, string cachePath, bool binary, string key, string salt, string[ ] columImportList=null, Action< Exception > onImport=null)

  (Pro Version only)
  Imports the GameDB from a server. Can be used in conjuction with the Golang server provided in the (Pro version) This allows over the air updating of GameDBs so new data can be pushed to clients without the need of releasing new versions of the clients. Downloaded GameDBs will be cached locally to avoid downloading again if they haven't changed. If they have change, only the diff between the original built-in GameDB and the uploaded GameDB will be downloaded.

Public Attributes

- Action OnDBLoaded = null

  OnDBLoaded allows callbacks to be added that are triggered if `notify = true` when importing or loading data. This allows code to deal with data potentially beind asynchronously loaded (when imported from a server) or hot reloaded via an Import or Reload from the GameDB Editor during run-time in editor.
- string Name => m_internal.Name

  Gets the name of the GameDB set when instantiating.

Properties

- string ScopeName = string.Empty `[get]`

  Gets the ScopeName of the GameDB set when creating it in the GameDB Editor.
- Logger Logger `[get, set]`

  Gets or sets the logger used for internal logs.

8.4.1 Detailed Description

The base class all GameDBs inherit from. The GameDB provides methods for importing data.

8.4.2 Member Function Documentation

8.4.2.1 Import() [1/2]

```
Exception GameDBLibrary.GameDBBase.Import (
            string jsonData,
            bool notify = true )
```

Imports JSON representing the GameDB.

Parameters

jsonData	A string representing the JSON format of the GameDB to import.
notify	if set to `true` the OnDBLoaded callback will be triggered for the GameDB (defaults to `true`).

Returns

An exception is returned if the GameDB fails to imprt

8.4.2.2 Import() [2/2]

```
Exception GameDBLibrary.GameDBBase.Import (
            string jsonData,
            string [] columImportList,
            bool notify = true )
```

Imports JSON representing the GameDB. Allows specifying only certain fields to be imported. Useful for limiting the memory used of the loaded GameDB.

Parameters

jsonData	A string representing the JSON format of the GameDB to import.
columImportList	An array of field names to import.
notify	if set to `true` the OnDBLoaded callback will be triggered for the GameDB (defaults to `true`).

Returns

An exception is returned if the GameDB fails to imprt

8.4.2.3 ImportFromServer()

```
RequestUpdater GameDBLibrary.GameDBBase.ImportFromServer (
            string serverHost,
            string downloadHost,
            string userID,
            string tag,
            string originalJSON,
            string cachePath,
            bool binary,
            string key,
            string salt,
            string [] columImportList = null,
            Action< Exception > onImport = null )
```

(Pro Version only)
Imports the GameDB from a server. Can be used in conjuction with the Golang server provided in the (Pro version) This allows over the air updating of GameDBs so new data can be pushed to clients without the need of releasing new versions of the clients. Downloaded GameDBs will be cached locally to avoid downloading again if they haven't changed. If they have change, only the diff between the original built-in GameDB and the uploaded GameDB will be downloaded.

Parameters

serverHost	The server host base address of the GameDB server to communicate with. For example: `https://mygamedbserver.com`
downloadHost	The download host base address of the GameDB to download. For example `https://s3.aws.com/mygamedb`
userID	A unique identified used to represent the client downloading the GameDB. Can be used for A/B testing GameDB versions
tag	The tag used to download the correct version of the GameDB
originalJSON	The original json the update is based off. This will be imported if there are no cached version or anything to download from the server
cachePath	The path to cache downloaded GameDBs too
binary	if set to `true` the GameDB will be loaded as a binary source (defaults to `false`) (`key` and `salt` required if loading binary)
key	the key used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`
salt	the salt used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`
columImportList	An array of field names to import.
onImport	A callback called when the data has be imported.

Returns

A RequestUpdater used to montior the request to the server.

8.4.3 Member Data Documentation

8.4.3.1 Name

```
string GameDBLibrary.GameDBBase.Name => m_internal.Name
```

Gets the name of the GameDB set when instantiating.

The name of the GameDB.

8.4.3.2 OnDBLoaded

```
Action GameDBLibrary.GameDBBase.OnDBLoaded = null
```

OnDBLoaded allows callbacks to be added that are triggered if `notify = true` when importing or loading data. This allows code to deal with data potentially beind asynchronously loaded (when imported from a server) or hot reloaded via an Import or Reload from the GameDB Editor during run-time in editor.

8.4.4 Property Documentation

8.4.4.1 Logger

```
Logger GameDBLibrary.GameDBBase.Logger  [get], [set]
```

Gets or sets the logger used for internal logs.

The logger.

8.4.4.2 ScopeName

```
string GameDBLibrary.GameDBBase.ScopeName = string.Empty  [get]
```

Gets the ScopeName of the GameDB set when creating it in the GameDB Editor.

The ScopeName

8.5 GameDBEditorLibrary.GameDBEditor Class Reference

GameDBEditor provides static methods for working with the GameDB Editor within the Unity Editor. Allowing things such as subscribing to events (load/save/promote) and programatically working with the editor.

Static Public Member Functions

- static bool LoadGameDB (string gameDBPath)

  Programatically load a GameDB in the editor. Useful if intergrating the GameDB Editor with other tools/editors.
- static bool SaveGameDB ()

  Programatically save the currently loaded GameDB in the editor.
- static void AddRowToTable (string table, string key, Dictionary< string, object > data)

  Allows the adding of a row to a GameDB table via an editor script. The GameDB needs to already be loaded via LoadGameDB method. After adding rows the GameDB needs to be saved via SaveGameDB
- static void RegisterSavedGameDBCallback (Action< string > onSaved)

  Allows a callback to be registered when a GameDB has been saved in the editor. Useful for updating other editors or systems when data has changed. The Scope Name of the saved GameDB is returned via the callback.
- static void RegisterRevisionPromotionCallback (Action< string, string, int > onPromotion)

  Allows a callback to be registered for when a revision of the GameDB is promoted on the GameDB Server. This is useful to trigger events such as causing live clients to update when a GameDB has been promoted. The Scope Name, tag and revision number are returned via the callback of the promoted GameDB.

8.5.1 Detailed Description

GameDBEditor provides static methods for working with the GameDB Editor within the Unity Editor. Allowing things such as subscribing to events (load/save/promote) and programatically working with the editor.

8.5.2 Member Function Documentation

8.5.2.1 AddRowToTable()

```
static void GameDBEditorLibrary.GameDBEditor.AddRowToTable (
            string table,
            string key,
            Dictionary< string, object > data )  [static]
```

Allows the adding of a row to a GameDB table via an editor script. The GameDB needs to already be loaded via LoadGameDB method. After adding rows the GameDB needs to be saved via SaveGameDB

Parameters

table	The table to add the row to.
key	The key of the row to add.
data	The dictionary representing the fields and data to add.

Exceptions

System.InvalidCastException	Thrown when data of an invalid type is added for a field.
System.ArgumentOutOfRangeException	Thrown when a field in the data doesn't exist in the table.

This example shows the code necessary to add a row to a table.

```
if (GameDBEditor.LoadGameDB("Test/testGameDB.json"))
{
    GameDBEditor.AddRowToTable(TestSchema.TableName, "testKey1", new Dictionary<string, object> {
        { TestSchema.FieldTest1, "test" },
        { TestSchema.FieldTest2, true }
    });

    GameDBEditor.AddRowToTable(TestSchema.TableName, "testKey2", new Dictionary<string, object> {
        { TestSchema.FieldTest1, "test2" },
        { TestSchema.FieldTest2, false }
    });
    GameDBEditor.SaveGameDB();
}
```

8.5.2.2 LoadGameDB()

```
static bool GameDBEditorLibrary.GameDBEditor.LoadGameDB (
            string gameDBPath )  [static]
```

Programatically load a GameDB in the editor. Useful if intergrating the GameDB Editor with other tools/editors.

Parameters

gameDBPath	The path to the GameDB to load.

Returns

`true`/`false` indicating success of the load operation.

8.5.2.3 RegisterRevisionPromotionCallback()

```
static void GameDBEditorLibrary.GameDBEditor.RegisterRevisionPromotionCallback (
            Action< string, string, int > onPromotion )  [static]
```

Allows a callback to be registered for when a revision of the GameDB is promoted on the GameDB Server. This is useful to trigger events such as causing live clients to update when a GameDB has been promoted. The Scope Name, tag and revision number are returned via the callback of the promoted GameDB.

Parameters

onPromotion	The callback triggered on a promotion.

8.5.2.4 RegisterSavedGameDBCallback()

```
static void GameDBEditorLibrary.GameDBEditor.RegisterSavedGameDBCallback (
            Action< string > onSaved )  [static]
```

Allows a callback to be registered when a GameDB has been saved in the editor. Useful for updating other editors or systems when data has changed. The Scope Name of the saved GameDB is returned via the callback.

Parameters

onSaved	The on saved.

8.5.2.5 SaveGameDB()

```
static bool GameDBEditorLibrary.GameDBEditor.SaveGameDB ( )  [static]
```

Programatically save the currently loaded GameDB in the editor.

Returns

`true`/`false` indicating success of the save operation.

8.6 GameDBLibrary.JsonPatch Class Reference

Implements the json patch spec at http://jsonpatch.com/ and passes all tests https://github.com/json-patch/json-patch-tests/blob/master/spec_tests.json Allows applying a json patch to a json string.

Public Member Functions

- string Patch (string originalJson, string patchJson)

  Patches the specified original json.

8.6.1 Detailed Description

Implements the json patch spec at http://jsonpatch.com/ and passes all tests https://github.com/json-patch/json-patch-tests/blob/master/spec_tests.json Allows applying a json patch to a json string.

8.6.2 Member Function Documentation

8.6.2.1 Patch()

```
string GameDBLibrary.JsonPatch.Patch (
            string originalJson,
            string patchJson )
```

Patches the specified original json.

Parameters

originalJson	The original json.
patchJson	The patch json.

Returns

The patched json string

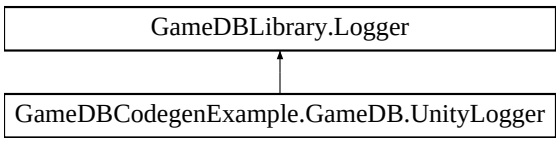
Exceptions

System.FormatException	
System.ArgumentException	

8.7 GameDBLibrary.Logger Class Reference

The logger class allows internal logging in the GameDB classes to be redirected to any destination required ie. Unity console, file on disk or terminal output.

Inheritance diagram for GameDBLibrary.Logger:



Public Member Functions

- virtual void Log (string message)

  Used to log general information from internal classes.
- virtual void LogError (string message)

  Used to log errors from internal classes.
- virtual void LogException (Exception e)

  Used to log exceptions from internal classes.

8.7.1 Detailed Description

The logger class allows internal logging in the GameDB classes to be redirected to any destination required ie. Unity console, file on disk or terminal output.

8.7.2 Member Function Documentation

8.7.2.1 Log()

```
virtual void GameDBLibrary.Logger.Log (
            string message )  [virtual]
```

Used to log general information from internal classes.

Parameters

message	The message to log.

8.7.2.2 LogError()

```
virtual void GameDBLibrary.Logger.LogError (
            string message )  [virtual]
```

Used to log errors from internal classes.

Parameters

message	The message to log.

8.7.2.3 LogException()

```
virtual void GameDBLibrary.Logger.LogException (
            Exception e )  [virtual]
```

Used to log exceptions from internal classes.

Parameters

e	The exception to log.

8.8 GameDBLibrary.Remote Class Reference

The Remote class provides methods to communicate with GameDB servers.

Static Public Member Functions

- static RequestUpdater GetLatestDeployment (string serverHost, string downloadHost, string scope, string tag, string checksum, string userID, string key, string salt, Action< Exception, string, int > callback)

  Gets the latest deployed GameDB for a particular tag.

8.8.1 Detailed Description

The Remote class provides methods to communicate with GameDB servers.

8.8.2 Member Function Documentation

8.8.2.1 GetLatestDeployment()

```
static RequestUpdater GameDBLibrary.Remote.GetLatestDeployment (
            string serverHost,
            string downloadHost,
            string scope,
            string tag,
            string checksum,
            string userID,
            string key,
            string salt,
            Action< Exception, string, int > callback )  [static]
```

Gets the latest deployed GameDB for a particular tag.

Parameters

serverHost	The server host base address of the GameDB server to communicate with. For example: `https://mygamedbserver.com`
downloadHost	The download host base address of the GameDB to download. For example `https://s3.aws.com/mygamedb`
scope	The scope name of the GameDB to check.
tag	The tag used to download the correct version of the GameDB
checksum	The checksum of the currently on disk GameDB.
userID	A unique identified used to represent the client downloading the GameDB. Can be used for A/B testing GameDB versions
key	the key used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`
salt	the salt used to encrypt the database via the GameDBLibrary.BinaryGameDB utilities. Required if using `binary = true`
callback	A callback called when the communication is complete. Returning information about the latest GameDB.

Returns

A RequestUpdater used to montior the request to the server.

8.9 GameDBLibrary.RequestUpdater Class Reference

RequestUpdater is a class that allows an update method to be hooked into an update loop, to allow constant updating of a requests asynchronous handling mechanisms

Public Member Functions

- void Update ()

  Updates the RequestUpdater. Call often to keep the associated callback receiving regular updates.

Public Attributes

- Action OnUpdate = null
  A callback to call each Update

8.9.1 Detailed Description

RequestUpdater is a class that allows an update method to be hooked into an update loop, to allow constant updating of a requests asynchronous handling mechanisms

8.9.2 Member Function Documentation

8.9.2.1 Update()

```
void GameDBLibrary.RequestUpdater.Update ( )
```

Updates the RequestUpdater. Call often to keep the associated callback receiving regular updates.

8.9.3 Member Data Documentation

8.9.3.1 OnUpdate

```
Action GameDBLibrary.RequestUpdater.OnUpdate = null
```

A callback to call each Update

8.10 GameDBLibrary.RowBase Class Reference

This is the base class for all Rows in a GameDB's tables. It provides accessors to the values store within.

Inherited by GameDBCodegenExample.Example, and GameDBEditorLibrary.Row.

Public Member Functions

- object GetValue (string field)
  Gets the value for a particular field.

8.10.1 Detailed Description

This is the base class for all Rows in a GameDB's tables. It provides accessors to the values store within.

8.10.2 Member Function Documentation

8.10.2.1 GetValue()

```
object GameDBLibrary.RowBase.GetValue (
            string field )
```

Gets the value for a particular field.

Parameters

field	The name of the field to retrieve.

Returns

The internally store value.

8.11 GameDBLibrary.ServerResponse Class Reference

ServerResponse handles parsing basic responses from the GameDB server

Static Public Member Functions

- static string HandleBasicResponse (IDownloadHandler response)
  Handles a basic response from the GameDBServer.

8.11.1 Detailed Description

ServerResponse handles parsing basic responses from the GameDB server

8.11.2 Member Function Documentation

8.11.2.1 HandleBasicResponse()

```
static string GameDBLibrary.ServerResponse.HandleBasicResponse (
            IDownloadHandler response )  [static]
```

Handles a basic response from the GameDBServer.

Parameters

response	The response object to parse.

Returns

`null` is returned if no error is found, otherwise an error message is returned

8.12 GameDBLibrary.Singleton< T > Class Template Reference

A helper base class for creating singletons. Used internally but provided for utility reasons.

Properties

- static T Instance [get]

  Returns the singleton instance (and instantiates it if not already instantiated)

8.12.1 Detailed Description

A helper base class for creating singletons. Used internally but provided for utility reasons.

Template Parameters

T	The type of the singleton class

Type Constraints

T* : *class

T* : *new()

8.12.2 Property Documentation

8.12.2.1 Instance

```
T GameDBLibrary.Singleton< T >.Instance  [static], [get]
```

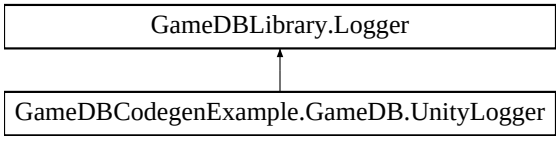
Returns the singleton instance (and instantiates it if not already instantiated)

The singleton instance of the inherited class.

8.13 GameDBCodegenExample.GameDB.UnityLogger Class Reference

(Unity only)
By default logging by the GameDB is logged via a GameDBLibrary.Logger. This allows the used logger to be replace if logging needs to be redirected. This class redirects logging to the Unity console via Debug.Log etc To replace the Logger set GameDBBase.Logger

Inheritance diagram for GameDBCodegenExample.GameDB.UnityLogger:



Additional Inherited Members

8.13.1 Detailed Description

(Unity only)
By default logging by the GameDB is logged via a GameDBLibrary.Logger. This allows the used logger to be replace if logging needs to be redirected. This class redirects logging to the Unity console via Debug.Log etc To replace the Logger set GameDBBase.Logger

See also

GameDBLibrary.Logger

Index